



شبكة الجزيرة الإعلامية
ALJAZEERA MEDIA NETWORK

VIZRT TEMPLATE BUILDER

External Command using Node.js Middleware

PREPARED BY ALJAZEERA CREATIVE SOLUTIONS



1. Overview

This document explains how Vizrt Template Builder can send commands to Viz Engine using a Node.js middleware server.

Instead of sending commands directly to Viz Engine, Template Builder sends an HTTP request to a Node.js service.

The middleware then forwards the appropriate command to Viz Engine, allowing templates to trigger actions such as animations or scene updates.

This approach provides a simple way to control Viz Engine from Template Builder while keeping the command logic centralized in a middleware service.



2. System Architecture

The system consists of three main components:

Template Builder

Runs the template script and sends an HTTP request when an action is triggered.

Node.js Middleware

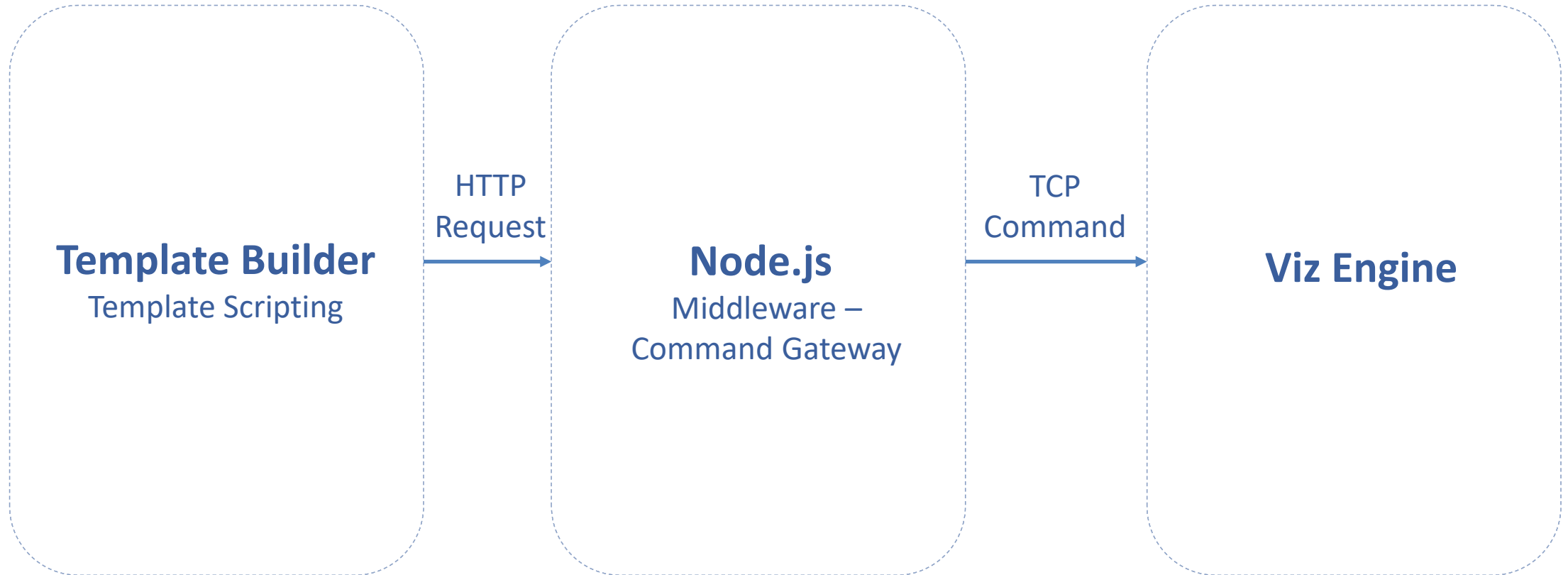
Receives the HTTP request and converts it into a Viz Engine command.

Viz Engine

Receives the command and executes the action inside the active scene.



2. System Architecture





3. Node.js Middleware

Node.js middleware service is used to forward commands from Template Builder to Viz Engine.

The middleware runs as a lightweight HTTP server and listens for incoming requests. When a request is received, the server forwards the command to the specified Viz Engine through a TCP connection.

Service Port

The middleware base url:

`http://10.202.13.133:3000`

API endpoint:

`http://10.202.13.133:3000/vizsend`



Required Parameters

Each request must include the following parameters:

Parameters	Description
host	Viz Engine IP address
port	Viz Engine command port
cmd	Viz Engine command

Command Index

Example: 0 RENDERER*TREE*\$box*TRANSFORMATION*POSITION GET

Index	Behavior
0	Returns the command response in the Vizrt console
-1	Silent mode (no response returned to the console)

Example Request

`http://10.202.13.133:3000/vizsend?host=10.200.13.133&port=6100&cmd=0%20RENDERER*STAGE*DIRECTOR*Default%20START`



4. Template Builder Integration

Template Builder can trigger Viz Engine commands by sending an HTTP request to the Node.js middleware. The request is sent to the middleware endpoint and includes the required parameters: host, port, and cmd.

Internal Scripting

Connection Configuration

The following variables define the connection parameters used to send commands to the middleware and Viz Engine.

```
const listener = "http://10.202.13.133:3000/vizsend"; //Middleware Endpoint  
const host = "10.202.13.133"; //Viz engine IP or host  
const port = 6100; //Viz engine port
```



Command Sender Function

The sendCommand function sends a Viz Engine command to the Node.js middleware.

```
async function sendCommand(host: string, port: Number, cmd: string, index: Number) {  
  const url =  
    listener +  
    "?host=" + host +  
    "&port=" + port +  
    "&cmd=" + encodeURIComponent(index + " " + cmd);  
  
  const res = await fetch(url);  
  const json = await res.json();  
  
  return json.response; //return command response from viz engine  
}
```




Example Usage

Commands triggered from Template Builder events `vizrt.onClick`:

```
vizrt.onClick = async (name: string) => {  
  switch (name) {  
    case "get":  
      const p = await sendCommand(host, port, "RENDERER*TREE*$box*TRANSFORMATION*POSITION GET ", 0);  
      console.log(p);  
      break;  
    case "set":  
      await sendCommand(host, port, "RENDERER*TREE*$box*TRANSFORMATION*POSITION SET 60 10 10", 0);  
      break;  
  }  
};
```



Example Usage

```
const listener = "http://10.202.13.133:3000/vizsend"; //Middleware Endpoint
const host = "10.202.13.133"; //Viz engine IP or host
const port = 6100; //Viz engine port

async function sendCommand(host: string, port: Number, cmd: string, index: Number) {
  const url =
    listener +
    "?host=" + host +
    "&port=" + port +
    "&cmd=" + encodeURIComponent(index + " " + cmd);

  const res = await fetch(url);
  const json = await res.json();

  console.log("VIZ RESPONSE:", json); //logging response
  return json.response; //return value response from viz engine
}

vizrt.onClick = async (name: string) => {

  switch (name) {

    case "get":
      const p = await sendCommand(host, port, "RENDERER*TREE*$box*TRANSFORMATION*POSITION GET ", 0);
      console.log(p);
      break;

    case "set":
      await sendCommand(host, port, "RENDERER*TREE*$box*TRANSFORMATION*POSITION SET 60 10 10", 0);
      break;

  }
};
```



5. Template Builder Integration (Using Script Units)

Script Units are stored in Viz Pilot's backend. This means they function as shared, centralized code:

When you create or update a Script Unit, the change is immediately available to all templates that import it.

Any template using that unit automatically picks up the new behavior as soon as the unit is saved, no redeployment of individual templates is required.



Example Usage

Script Units

```
// @unit/externalcommand

let listenersocket: any = null;

export function createConnection(listener: string) {
  listenersocket = listener
  return listenersocket
}

export async function sendCommand(host: string, port: Number, cmd: string, index: Number) {
  const url =
    listenersocket +
    "?host=" + host +
    "&port=" + port +
    "&cmd=" + encodeURIComponent(index + " " + cmd);

  const res = await fetch(url);
  const json = await res.json();

  return json.response; //return value response from viz engine
}
```



Example Usage

Template Builder script

```
import {createConnection, sendCommand } from '@unit/externalcommand';

const listener = "http://10.202.13.133:3000/vizsend"; //Middleware Endpoint
const host = "10.202.13.133"; //Viz engine IP or host
const port = 6100; //Viz engine port

createConnection(listener); //call function from script units

vizrt.onClick = async (name: string) => {

  switch (name) {

    case "get":
      const p = await sendCommand(host, port, "RENDERER*TREE*$box*TRANSFORMATION*POSITION GET ", 0);
      console.log(p);
      break;

    case "set":
      await sendCommand(host, port, "RENDERER*TREE*$box*TRANSFORMATION*POSITION SET 60 10 10", 0);
      break;

  }
};
```



Create, Modify and Delete Script Units

To manage Script Units, click the Script Units icon to open the list:

The screenshot shows the 'Script Units' interface. On the left, there is a list of script units with checkboxes: 'catchdata', 'datareader', 'datareaderv2', 'externalcommand' (checked), 'externalcommandv2', and 'roundeddecimal'. Above this list is a search bar labeled 'Find script unit'. On the right, the 'Template Script' editor is open, displaying a JavaScript-like script. The script starts with an import statement for 'externalcommand' and defines several variables and functions. At the bottom, a green checkmark indicates 'No syntax errors'.

```
1 import {sendCommand, createConnection} from '@unit/externalcommand'
2
3 const vf = vizrt.fields;
4 //host is string
5 let host = vf.$txt_IP.value;
6 //post is number/integer
7 let port = Number(vf.$txt_port.value);
8 //container name
9 let container = vf.$CONTAINER_NAME.value;
10
11 //ip socket listener
12 const listener = "http://10.202.13.133:3000/vizsend";
13 createConnection(listener);
14
15 function getFieldValue(value: any): string {
16   if (value == null) return "";
17   return String(value);
18 }
19
20 function parsePos(value: any){
21   const pos = value.split(" ");
```

Template Example

Concept :UTILITY

Template :SEND_COMMAND_V2